

AI Interview Platform - Interview Prep

Use this as your 15-minute deep-dive script. It matches the current implementation: Next.js App Router, API routes, Prisma, MySQL/TiDB, Upstash Redis, and Google Gemini.

Q1. Walk Me Through The Platform End To End

Strong opening answer

"The AI Interview Platform is a Next.js App Router application backed by API routes, Prisma, MySQL/TiDB, Upstash Redis, and Google Gemini. The user starts an anonymous interview session by choosing a role and difficulty. I identify the user with an `ai_uid` cookie, enforce a Redis daily quota, create an `InterviewSession` in the database, serve one unattempted question at a time, and when the user submits an answer I validate it, lock the submission path, check the evaluation cache, call Gemini only on a cache miss, persist the attempt and structured evaluation, and return the result to the UI."

Data flow, step by step

1. User selects `role` and `difficulty` on the home page.
2. Clicking Start Session calls `POST /api/session/start` with `{ role, difficulty }`.
3. The start route reads the anonymous `ai_uid` cookie. If missing, it creates a UUID and sets it as an HTTP-only cookie.
4. Zod validates `role` and `difficulty`: `"Easy" | "Medium" | "Hard"`.
5. The route calls `consumeDailySessionQuota(uid)`. Redis increments the user's daily key. If count is above 3, the route returns HTTP `429` and creates no session.
6. If quota is allowed, Prisma upserts `User` and creates an `InterviewSession` with `userId`, `role`, `difficulty`, and `status = ACTIVE`.
7. The frontend receives `{ sessionId, quota }` and redirects to `/session/{sessionId}`.
8. The session page calls `GET /api/session/{sessionId}/question`.
9. The question route verifies the cookie, session ownership, and active status, finds attempted question ids, and selects the first unattempted question matching role and difficulty.
10. The route returns `question`, `total`, and `questionNumber`.
11. User submits answer through `POST /api/session/{sessionId}/attempt` with `{ questionId, answerText }`.
12. The attempt route validates body, user cookie, session ownership/status, and question existence.
13. The route acquires Redis lock `lock:attempt:{uid}:{sessionId}` with `SET key "1" NX EX 30`. If locked, it returns HTTP `429`.
14. Once locked, it creates an `Attempt` row with session id, question id, and answer text.
15. It computes `sha256(`${PROMPT_VERSION}|${questionId}|${answerText}`)`. Conceptually this is question plus answer plus prompt version; `questionId` is the stable question identity.
16. It checks Redis key `cache:eval:{hash}`.
17. On cache hit, Gemini is skipped and the cached evaluation is reused.
18. On cache miss, `evaluateWithRetry` calls `evaluateAnswer` in `src/lib/gemini.ts`.
19. Gemini returns JSON, the server parses and validates it with Zod, caches it for 14 days, and updates the `Attempt` row.
20. The API returns `{ attemptId, createdAt, cacheHit, evaluation }`. The UI displays score, feedback, ideal answer, tags, and HIT/MISS.

Where exactly Gemini gets called

Gemini is called only inside `POST /api/session/[id]/attempt`, and only on a cache miss. The route calls `evaluateWithRetry`, which calls `evaluateAnswer` in `src/lib/gemini.ts`. Inside that function, `ai.models.generateContent`

sends role, difficulty, question prompt, and candidate answer to `gemini-2.5-flash` .

```
ai.models.generateContent({
  model: "gemini-2.5-flash",
  contents: prompt,
  config: {
    systemInstruction,
    responseMimeType: "application/json",
    responseJsonSchema: zodToJsonSchema(EvaluationSchema),
    temperature: 0.3
  }
})
```

Structured feedback fields

```
{
  score: number,      // integer 0 to 10
  feedback: string,    // concise actionable feedback
  idealAnswer: string, // best short answer
  tags: string[]       // improvement/topic tags
}
```

The project enforces this by asking Gemini for JSON matching a schema and then validating the returned JSON with `EvaluationSchema.safeParse` .

If Gemini fails mid-session

The attempt route retries once after 300ms. If Gemini still fails, returns invalid JSON, or fails schema validation, the API returns HTTP `502` . The `finally` block releases the Redis lock. The frontend stays on the same question, shows the error, and allows retry.

Current implementation detail: the `Attempt` row is created before evaluation. If Gemini fails after that, the database can contain an unevaluated attempt. The history page handles that as "Not evaluated yet." A production improvement would be an explicit evaluation status or idempotency key.

Q2. Redis-Backed 3 Sessions/Day Rate Limiting

Strong opening answer

"I enforce the daily session quota at session creation time, before writing the session to the database. Each user has a Redis key scoped by user id and IST calendar date. I call atomic `INCR` on that key, set its expiry to the next IST midnight if it does not already have one, and allow the request only if the resulting count is less than or equal to 3."

Exact key and commands

```
quota:sessions:{userId}:{YYYY-MM-DD}
```

```
INCR quota:sessions:{userId}:{date}
TTL quota:sessions:{userId}:{date}
EXPIRE quota:sessions:{userId}:{date} {secondsUntilNextIstMidnight} # only if TTL == -1
```

Then the route computes `allowed = count <= 3` and `remaining = Math.max(0, 3 - count)` . Counts 1, 2, and 3 are allowed; count 4 returns HTTP `429` .

Concurrent requests

Redis `INCR` is atomic, so two simultaneous session starts cannot both read the same old value. If the current count is 2 and two requests arrive together, one `INCR` returns 3 and is allowed; the other returns 4 and is denied.

Near reset, the date suffix naturally moves traffic to a new IST-date key after midnight. Redis still processes commands atomically. If the old key has expired, the first `INCR` creates the new key at 1; the next returns 2.

Where distributed locking fits

Be precise: quota uses atomic `INCR`; the distributed lock is for answer submission, not for the daily session counter.

```
lock:attempt:{uid}:{sessionId}
SET lock:attempt:{uid}:{sessionId} "1" NX EX 30
DEL lock:attempt:{uid}:{sessionId}
```

`NX` means only one request can acquire the lock. `EX 30` is a safety expiry. This prevents duplicate browser submits from creating multiple simultaneous Gemini calls for the same user/session.

Honest improvement if drilled

"One improvement would be to combine `INCR` and first-time `EXPIRE` in a Lua script or transaction. The current code mitigates missing TTL by checking `TTL == -1` and setting it, and because the key includes the date, stale old-date keys do not affect the next day's quota."

Q3. SHA-256 Evaluation Caching

Strong opening answer

"I cache evaluations so the same answer to the same question does not trigger another Gemini call. The cache key is a SHA-256 hash. Conceptually it is based on the question and answer pair, plus a prompt version. In the current code, I hash `PROMPT_VERSION`, `questionId`, and `answerText`, where `questionId` is the stable identity of the question."

Exact hash input

```
const evalHash = sha256(`${PROMPT_VERSION}|${questionId}|${answerText}`);
cache key = cache:eval:{sha256Hash}
TTL = 14 days
```

Cache hit vs miss

Hit: create hash, read Redis, reuse evaluation, skip Gemini, update current attempt, return `cacheHit: true`.

Miss: create hash, Redis returns null, call Gemini, validate JSON, store `cache:eval:{hash}` with `EX 1209600`, update attempt, return `cacheHit: false`.

Invalidation risks

- Question prompt changes while `questionId` stays the same.
- Scoring rubric or system instruction changes without bumping `PROMPT_VERSION`.
- Model version changes and old evaluations should not be reused.
- Role/difficulty context changes in a way that should affect scoring.
- Output fields or tag meaning changes.

Good answer: "I included `PROMPT_VERSION` so I can invalidate old evaluations when the prompt or rubric changes. A further improvement would be to include question prompt text, model name, role, difficulty, or a question content version in the hash."

Cost reduction answer

"I did not run a full production cost study yet. I implemented the instrumentation needed to observe it: the attempt API returns `cacheHit`, the UI displays HIT/MISS, and attempts store `evalHash`. In production I would track cache-hit ratio as `cache_hits / total_evaluations`, then estimate saved cost as `cache_hits * average Gemini cost per evaluation`."

Short Answers To Memorize

- **Architecture:** Next.js App Router and API routes, Prisma with MySQL/TiDB, Upstash Redis for quota/cache/locks, and Gemini for structured answer evaluation.
 - **Submit flow:** Validate cookie/body/session/question, take Redis lock, create attempt, compute SHA-256 cache key, reuse cache on hit, call Gemini on miss, validate JSON, cache it, update attempt, return feedback, release lock.
 - **Quota:** `quota:sessions:{userId}:{IST-date}`; atomic `INCR`; expiry to next IST midnight; allow only counts 1 through 3.
 - **Locking:** `lock:attempt:{uid}:{sessionId}` acquired with `SET NX EX 30`.
 - **Caching:** `cache:eval:{sha256(PROMPT_VERSION|questionId|answerText)}`; hits skip Gemini; misses call Gemini and store evaluation for 14 days.
-

Senior-Sounding Phrases

- "I separate durable state from ephemeral coordination: Prisma stores sessions and attempts; Redis handles quota counters, cache entries, and short-lived locks."
- "Gemini is behind the attempt API, not directly exposed to the client."
- "I validate external input twice: request bodies with Zod, and AI output with a Zod schema."
- "The cache is versioned by prompt version because AI evaluation is not just a function of text; it is also a function of the rubric."
- "For quota, atomic INCR solves the concurrency problem. For answer submission, SET NX EX prevents duplicate expensive side effects."